PRACTICAL 5: LDA TOPIC MODELING

```
!pip install -q gensim pyLDAvis nltk scikit-learn
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.8/27.8 MB 70.6 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.6/2.6 MB 74.5 MB/s eta 0:00:00
```

```
import nltk
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

```
import re
import numpy as np
import matplotlib.pyplot as plt
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
from gensim import corpora
from gensim.models import LdaModel
from gensim.models.coherencemodel import CoherenceModel
import pyLDAvis
import pyLDAvis.gensim_models as gensimvis
```

```
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is dep
  return datetime.utcnow().replace(tzinfo=utc)
```

```
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
```

```
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is dep
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is dep
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is dep
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is dep
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is dep
  return datetime.utcnow().replace(tzinfo=utc)
```

```
# --- Step 1: Load a sample corpus (20-Newsgroups) --
from sklearn.datasets import fetch_20newsgroups
categories = ['sci.space', 'rec.sport.hockey', 'talk.politics.mideast', 'comp.graphics']
newsgroups = fetch_20newsgroups(
    subset='train', categories=categories,
    remove=('headers', 'footers', 'quotes')
)
documents = newsgroups.data[:500]   # limit for speed on Colab
print(f"Loaded {len(documents)} documents")
print(documents[0][:300])
```

```
Loaded 500 documents
: 8~> I require BGI drivers for Super VGA Displays and Super XVGA Displays. Does
: 8~> anyone know where I could obtain the relevant drivers ? (FTP sites ??)

:       I would like to know too!

: Regards,
: Dominic

garbo.uwasa.fi (or one of its many mirrors) has a file
called "svgabg40" in the programm
```

```
# --- Step 2: Preprocessing (tokenization + stop word removal + lemmatization) --
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
```

```
def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', ' ', text)          # remove numbers/punctuation
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t) for t in tokens
              if t not in stop_words and len(t) > 3]
    return tokens
processed_docs = [preprocess(doc) for doc in documents]
processed_docs = [doc for doc in processed_docs if len(doc) > 5]  # drop near-empty docs
print("Sample tokens:", processed_docs[0][:15])
```

```
Sample tokens: ['require', 'driver', 'super', 'display', 'super', 'xvga', 'display', 'anyone', 'know', 'could', 'obtain', 'r
```

```
# --- Step 3: Build dictionary and bag-of-words corpus --
dictionary = corpora.Dictionary(processed_docs)
dictionary.filter_extremes(no_below=5, no_above=0.5)   # remove rare & overly common words
bow_corpus = [dictionary.doc2bow(doc) for doc in processed_docs]
```

```
# --- Step 4: Train the LDA model --
NUM_TOPICS = 4
lda_model = LdaModel(
    corpus=bow_corpus,
    id2word=dictionary,
    num_topics=NUM_TOPICS,
    random_state=42,
    passes=10,
    iterations=100,
    alpha='auto',
    per_word_topics=True
)
```

```
# --- Step 5: Print the topics and their top words --
print("\n=== Discovered Topics ===")
for idx, topic in lda_model.print_topics(num_words=8):
    print(f"Topic {idx}: {topic}\n")
```

```

=== Discovered Topics ===
Topic 0: 0.016*"people" + 0.016*"said" + 0.016*"know" + 0.013*"armenian" + 0.011*"would" + 0.011*"like" + 0.008*"time" + 0.0

Topic 1: 0.018*"israel" + 0.017*"israeli" + 0.015*"would" + 0.009*"arab" + 0.007*"year" + 0.007*"jew" + 0.006*"armenian" + 0

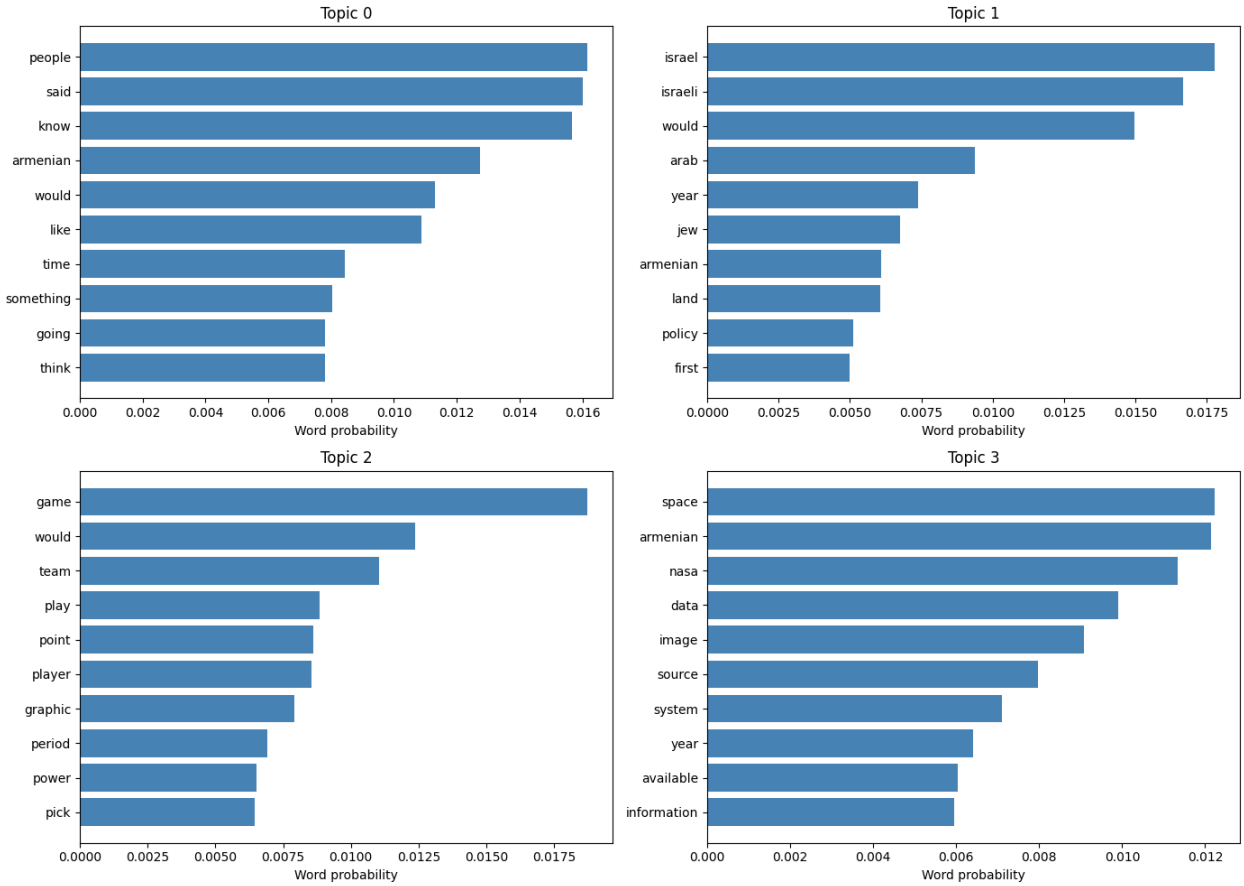
Topic 2: 0.019*"game" + 0.012*"would" + 0.011*"team" + 0.009*"play" + 0.009*"point" + 0.009*"player" + 0.008*"graphic" + 0.0

Topic 3: 0.012*"space" + 0.012*"armenian" + 0.011*"nasa" + 0.010*"data" + 0.009*"image" + 0.008*"source" + 0.007*"system" +
```

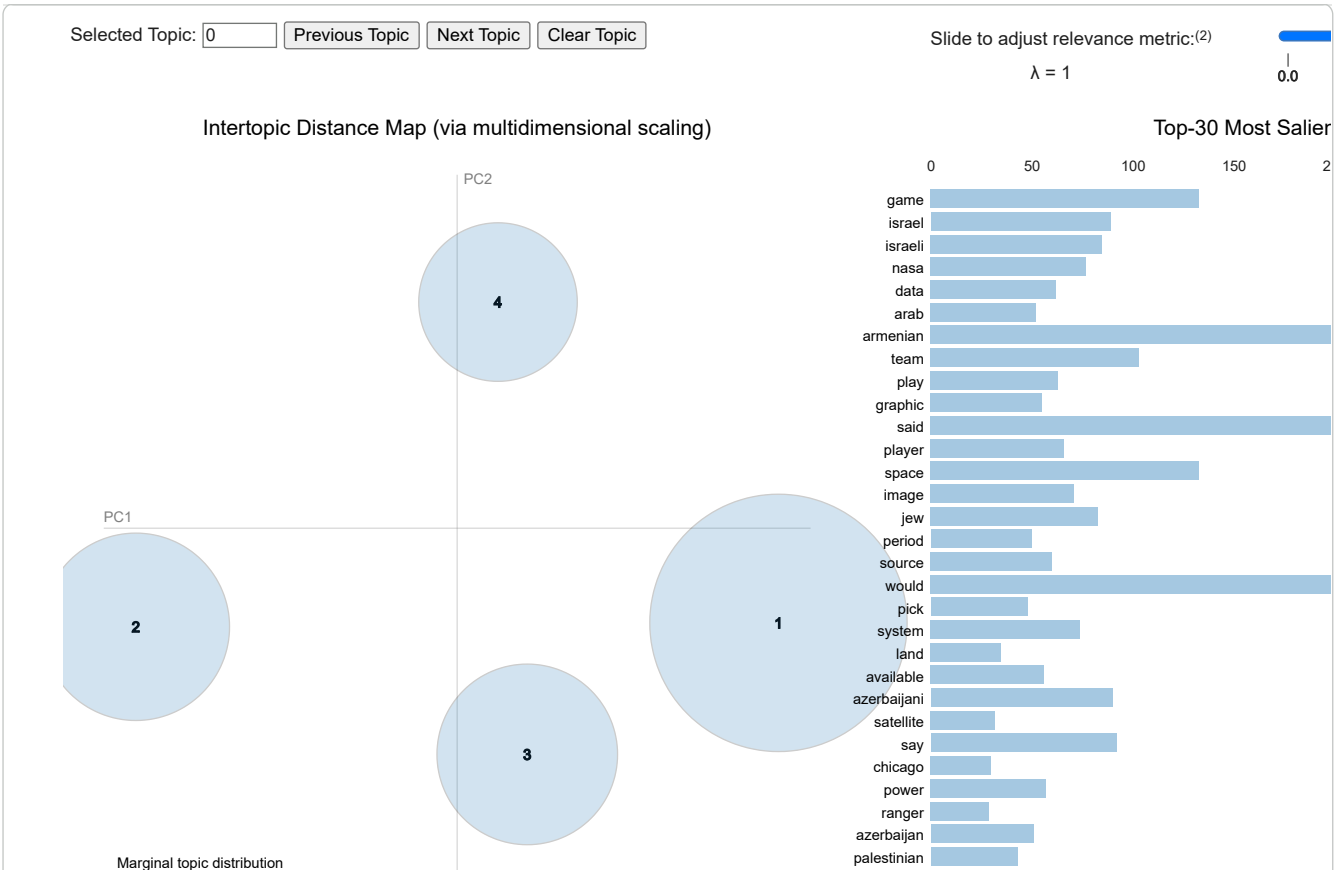
```
# --- Step 6: Evaluate topic quality with Coherence Score --
coherence_model = CoherenceModel(
    model=lda_model, texts=processed_docs,
    dictionary=dictionary, coherence='c_v'
)
print(f"Coherence Score (c_v): {coherence_model.get_coherence():.4f}")
```

```
Coherence Score (c_v): 0.4115
```

```
# --- Step 7: Visualize topics — bar charts of top words per topic --
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()
for i in range(NUM_TOPICS):
    words_probs = lda_model.show_topic(i, topn=10)
    words = [w for w, _ in words_probs][::-1]
    probs = [p for _, p in words_probs][::-1]
    axes[i].barh(words, probs, color='steelblue')
    axes[i].set_title(f"Topic {i}")
    axes[i].set_xlabel("Word probability")
plt.tight_layout()
plt.savefig('lda_topics_barchart.png', dpi=150)
plt.show()
```




```
# --- Step 8: Interactive visualization with pyLDAvis --
pyLDAvis.enable_notebook()
vis = gensimvis.prepare(lda_model, bow_corpus, dictionary)
pyLDAvis.save_html(vis, 'lda_visualization.html')
vis   # renders inline in Colab/Jupyter
```


Selected Topic: 0
Previous Topic
Next Topic
Clear Topic
Slide to adjust relevance metric:(2)
λ = 1
0.0
Intertopic Distance Map (via multidimensional scaling)
Top-30 Most Salier
PC2
PC1
4
2
1
3
Marginal topic distribution
0
50
100
150
2
game
israel
israeli
nasa
data
arab
armenian
team
play
graphic
said
player
space
image
jew
period
source
would
pick
system
land
available
azerbaijani
satellite
say
chicago
power
ranger
azerbaijan
palestinian